Privacy-Preserving Learning-Augmented Data Structures

Prabhav Goyal, Vinesh Sridhar, Wilson Zheng

University of California, Irvine

Dictionary Problem

- Dictionaries are a fundamental class of data structures that support updates and retrievals of key-value pairs.
- Data structures that support inexact searches, such as predecessor and range queries, achieve retrieval times of at best $O(\log n)$.
- Unfortunately, these structures often ignore information about the keys' access frequencies.

Biased Search Trees $\rightarrow$ Learning

- BST give distribution sensitive guarantees if weights w_i are known (in practice, rarely known).
- Each key k_i is assigned a positive weight w_i (e.g., access frequency estimate).
- Search time: $O(\log(W/w_i))$ where $W = \sum_j w_j$.
- ML model provides an **estimate** f_i of true access probability.
- **Key Idea:** treat ML predictions as weights (black-box) to combine classical distribution bounds with learned predictions.

Learning-Augmented Data Structures (LADS)

- ML Oracle provides estimates $f_i \in (0, 1]$ for keys k_i, typically $\sum_i f_i \leq 1$.
- A LADS uses f_i to layout so frequent keys are faster (better than $O(\log n)$ for searches).
- Works as a black-box transformation on many biased DS: treaps, zip-zip trees, skip-lists.
- **Challenge:** Retain good performance for frequent keys and adversarial predictions. Also have privacy guarantees.

Consistency

Intuition: Frequent keys (large f_i) are faster than rare keys, giving good performance.

Definition

A LADS is consistent w.r.t. predicted frequencies $\{f_i\}$ if the lookup of key k_i costs $O(\log(1/f_i))$.

- Lin et al. [1] coins this with the First Learning-Augmented Search Tree (based off Treap)

Robustness

Intuition: Protects against potential ML model failures or adversarially chosen predictions.

Definition

A LADS is robust if the worst-case lookup cost is $O(\log n)$.

- Zeynali et al. [2] - RobustSL (based on Skip List)
  - Achieves consistency and robustness against adversarial predictions.
  - Supports dynamic updates but (as we show) is not history independent under its amortized updates.
  - Now we operate in linear time, not constant.

Limitations of Prior Work

- **Ad-hoc robustness:** RobustSL obtains robustness via a skip-list–specific amortized rebuild mechanism — not a simple, general transform.
- **History leakage & Need for Strong-Privacy:** Memory layouts of prior LADS reflect operation history which yields historical leakage.
- What about Splay Trees? They satisfy consistency + robustness.
  - Their structure directly reflects past accesses.
  - Therefore, also could leak historical information.

Our Goals and Contributions

- **Goal:** Simple, portable methods that (i) preserve learned speed, (ii) guarantee worst-case $O(\log n)$, (iii) prevent layout leakage, (iv) support dynamic updates.

- **Problems $\rightarrow$ Contributions**
  - Ad-hoc robustness $\Rightarrow$ **Thresholding**: black-box transform on f_i (applies to any biased DS).
  - Rebuild/rotation-based leakage $\Rightarrow$ **Weakly H.I. Update Scheme**: randomized canonical-cutoff to hide rebuilds.
  - Need for strong privacy in dynamic settings $\Rightarrow$ **Pairing**: maintain learned + non-learned HI copies for strong H.I.

Thresholding — Motivation

- **Problem**: Extremely small predicted frequencies ($f_i \ll 1/n$) create vulnerability.
  - Under adversarial predictions a consistent LADS may degrade to large depths / linear traversals.
  - Example: A truly hot key with real frequency $\frac{1}{2}$ but predicted $f_i \approx 2^{-n} \Rightarrow$ lookup cost may become linear.
- Prior robust approaches: group small-frequency keys into a separate non-learned structure with $O(\log n)$ cost.
- Our idea: modify the frequencies themselves so that any consistent LADS becomes robust with minimal change.
  - Simple, black-box approach.

Threshold Frequency Scheme

Definition

Given predicted frequencies f_i (with $\sum_i f_i \leq 1$) and total keys n, define

$$f_i' \;:=\; \max\{\, f_i/2 \;,\; 1/(2n) \,\}.$$

Insert each key using f_i' (instead of f_i) into the LADS.

Theorem

Any consistent learning-augmented data structure $\mathscr{D}$ can be made robust by applying the threshold frequency scheme.

Thresholding - Proof

Intuition: Cap how small a frequency can be (worst-case depth $O(\log n)$) while preserving $O(\log(1/f_i))$ for frequent keys up to small constant factors.

Proof Sketch.

- **Feasibility:** At most n keys are raised by at most $1/(2n)$, so

$$\sum_i f_i' \leq \frac{1}{2} \sum_i f_i + n \cdot \frac{1}{2n} \leq \frac{1}{2} + \frac{1}{2} = 1.$$

Thus the modified frequencies form a valid distribution.

□

Thresholding: Practical Implications

- **Guarantees consistency:**

$$f_i' \geq \frac{f_i}{2} \quad \Rightarrow \quad O\big(\log(1/f_i')\big) = O\big(\log(2/f_i)\big) = O\big(\log(1/f_i)\big)$$

(only small additive constant-factor)

- **Guarantees robustness:**

$$f_i' \geq \frac{1}{2n} \quad \Rightarrow \quad O(\log n) \text{ worst-case depth}$$

- **Dynamic** n: Maintain a canonical cutoff N (weak-HI update scheme):

$$f_i' = \max\{f_i/2, 1/(2N)\}$$

ensures robustness and history independence even as the set grows.

History Independence (Intuition)

A data structure's memory layout should reveal only the current set of key-value contents, not the sequence of operations that produced them.

- Protects against information leakage after breaches.
- Two Standard Notions: **strong** and **weak** history independence.

History Independence (Formal)

Consider two states A (initial) and B (final), and two operational sequences X, Y that take $A \to B$.

Definition (Strong History Independence)

A data structure is strongly history independent if for any X, Y with the same final state B, the internal memory representation induced by X and Y are **indistinguishable**.

Definition (Weak History Independence)

A data structure is weakly history independent if the above indistinguishability property holds only when A is a canonical initial state (e.g., $A = \emptyset$).

RobustSL Update Bucket Scheme

- Maintain two numbers: n = current size, N = cutoff parameter (initially $n = 0,\ N = 4$).
- Insert: if n becomes N, set $N \leftarrow N^2$ and rebuild (cost $O(n \log n)$).
- Delete: if n becomes $N^{1/4}$, set $N \leftarrow \sqrt{N}$ and rebuild.
- Amortized $O(\log n)$ per update with high probability, but rebuilding decisions leak history via N changes.

Problem with Existing Dynamic Robust LADS

- **RobustSL** achieves robustness via an amortized rebuild scheme that depends on a cutoff parameter N.
- Because N is changed via rebuilds, two different operation sequences can lead to the same state but different internal layouts / distributions.
- Counterexample (succinct): Let $c := N - n$.
  - Sequence X: Insert c identical keys then delete 1.
  - Sequence Y: Insert $c - 1$ identical keys.
  - Both lead to same final contents, but X changed N while Y did not $\Rightarrow$ distinguishable layouts.
- Therefore, RobustSL is not History Independent.

Our Approach: Make Updates History-Independent

- Use Hartline et al.'s [3] canonical-cutoff idea:
  - Associate each integer n with a canonical range for N so that a fixed n maps to a canonical N.
  - Avoid multiple valid N values per n.
- Introduce private randomness to the cutoff choice so adversary cannot trigger rebuilds deterministically.
- Rebuilds occur with small probability $\Theta(1/n)$ per operation, so the expected cost per operation remains $O(\log n)$.

Weakly H.I. Update Scheme — Analysis

Theorem

There exists a weakly history independent, consistent, and robust learning-augmented data structure that supports $O(\log n)$-time dynamic updates in expectation.

Proof Sketch.

1. Rebuild with probability $O(1/n)$ per update.
2. Each rebuild costs $O(n \log n)$
3. Expected rebuild cost per operation:
   $O((n \log n) \cdot (1/n)) = O(\log n)$.

Pairing

- Build a **paired data structure** $\mathscr{D}_P$ consisting of two coordinated copies:
  - $\mathscr{D}_C$: a consistent, strongly history independent learned (biased) structure.
  - $\mathscr{D}$: a strongly history independent, non-learned structure (supports inexact queries).
- Maintain **tandem updates** (insert/delete in both structures).
- **Query strategy**: search $\mathscr{D}_C$ for $\gamma \log n$ steps; if not found, fall back to $\mathscr{D}$.[1]
- **Guarantees**: $O(\min\{\log(1/f_i),\ \log n\})$ lookup; strong history independence preserved dynamically.

[1] Choice of γ: tradeoff constant; larger γ reduces fallback frequency but increases worst-case learned-search work.

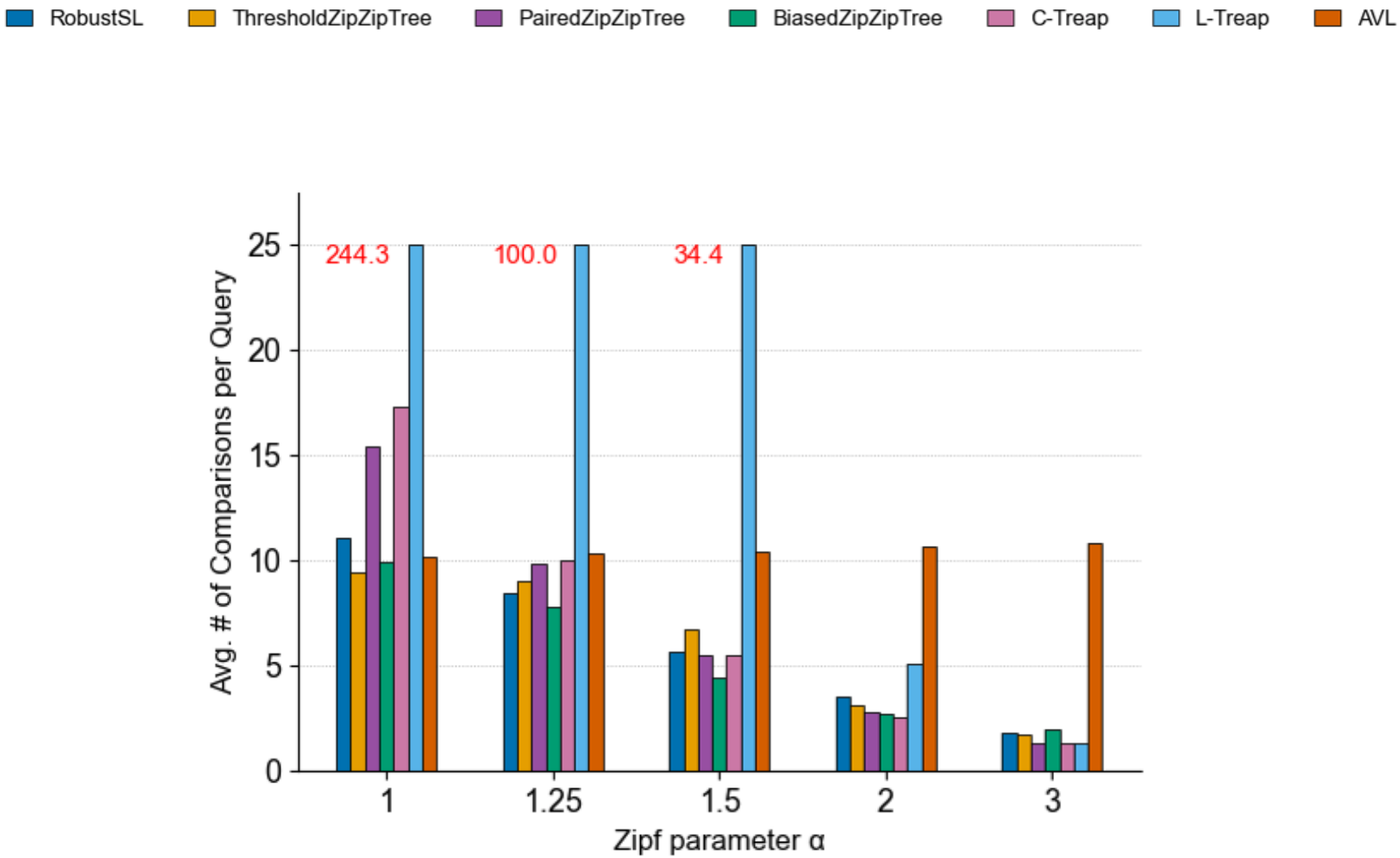
Pairing Costs & Tradeoffs

- **Space:** $\approx 2n$ (two representations/copies); may be pointer-shared copy to reduce duplication.
- **Latency:** $O(\min\{\log(1/f_i),\ \log n\})$ — frequent keys acquire learned speed; rare keys pay up to factor $\approx 1+\gamma$ relative to thresholded DS in practice.
- **Updates:** In the worst-case, takes linear time because the biased structure could be unbalanced.
- **History independence:** Strong

Experimental Setup (Static Regime)

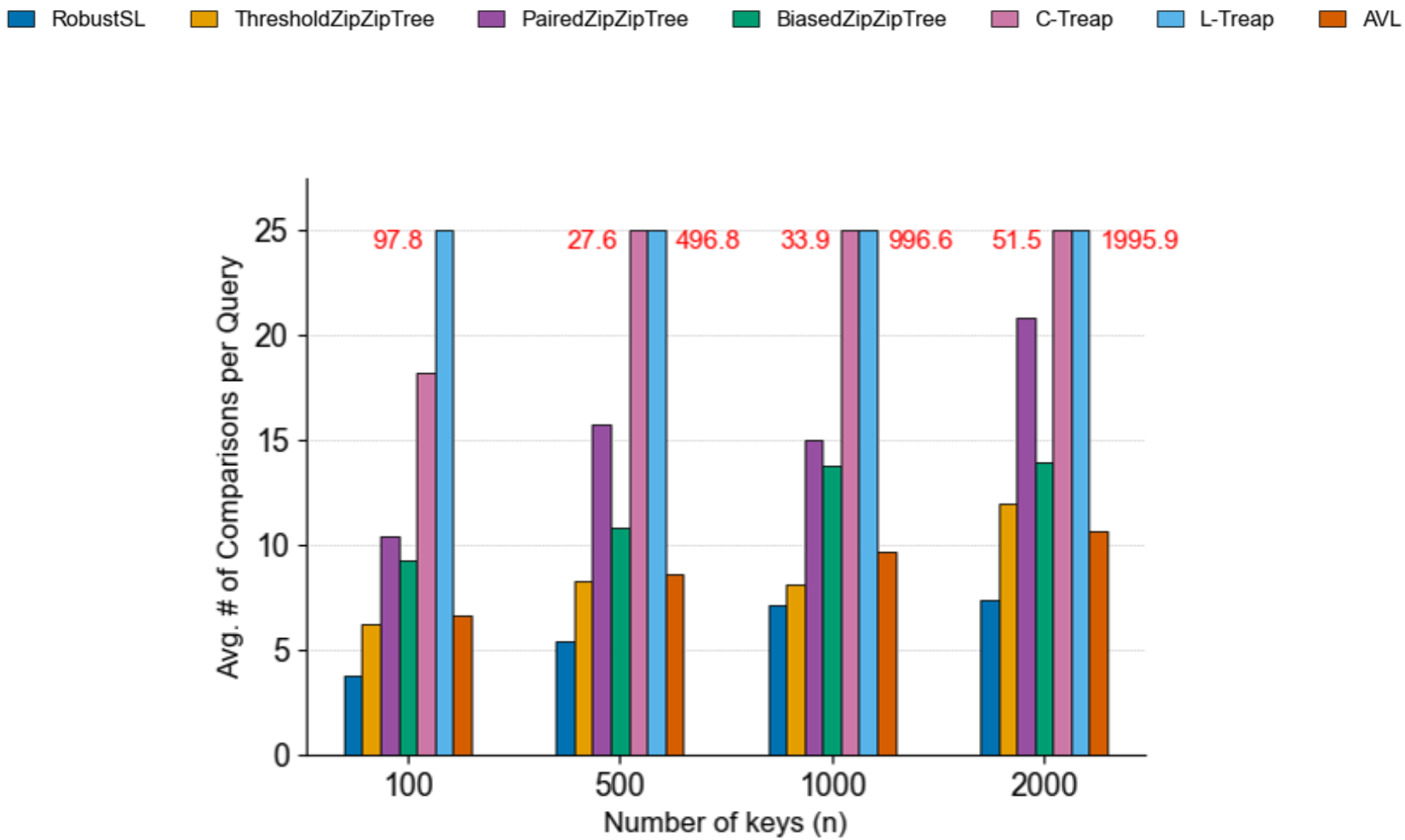
- Comparisons: RobustSL, Biased Zip-Zip, Thresholded Zip-Zip, Paired Zip-Zip ($\gamma = 1$), L-Treap, C-Treap, AVL.
- Workloads:
  - **Zipfian:** Zipf Parameter Test
    - rank i frequency $\propto 1/i^{\alpha}$; vary α ($n = 2000$).
    - higher α = more skew
  - **Noisy Zipfian:**
    - Adversarialized ranks with δ (used $\alpha = 2,\ \delta = 0.9$)
    - $\hat{i} \leftarrow i \times (1 - \delta),\ \delta \times (n - i + 1) \quad \delta \in [0,\ 1]$
    - Insert keys under ranks $\hat{i}$ (adversarial ordering), but query under true ranks i.
  - **Inverse-Power:**
    - frequency $\propto 1/\alpha^{i}$ (extreme small probs; $\alpha = 1.01,\ \delta = 0.9$).
    - Exponentially decaying probabilities; stresses tiny-f behavior.
- **Protocol**: Insert keys under (possibly adversarial) ranks; run queries; measure average comparisons (and node counts).

Zipfian Parameter Test



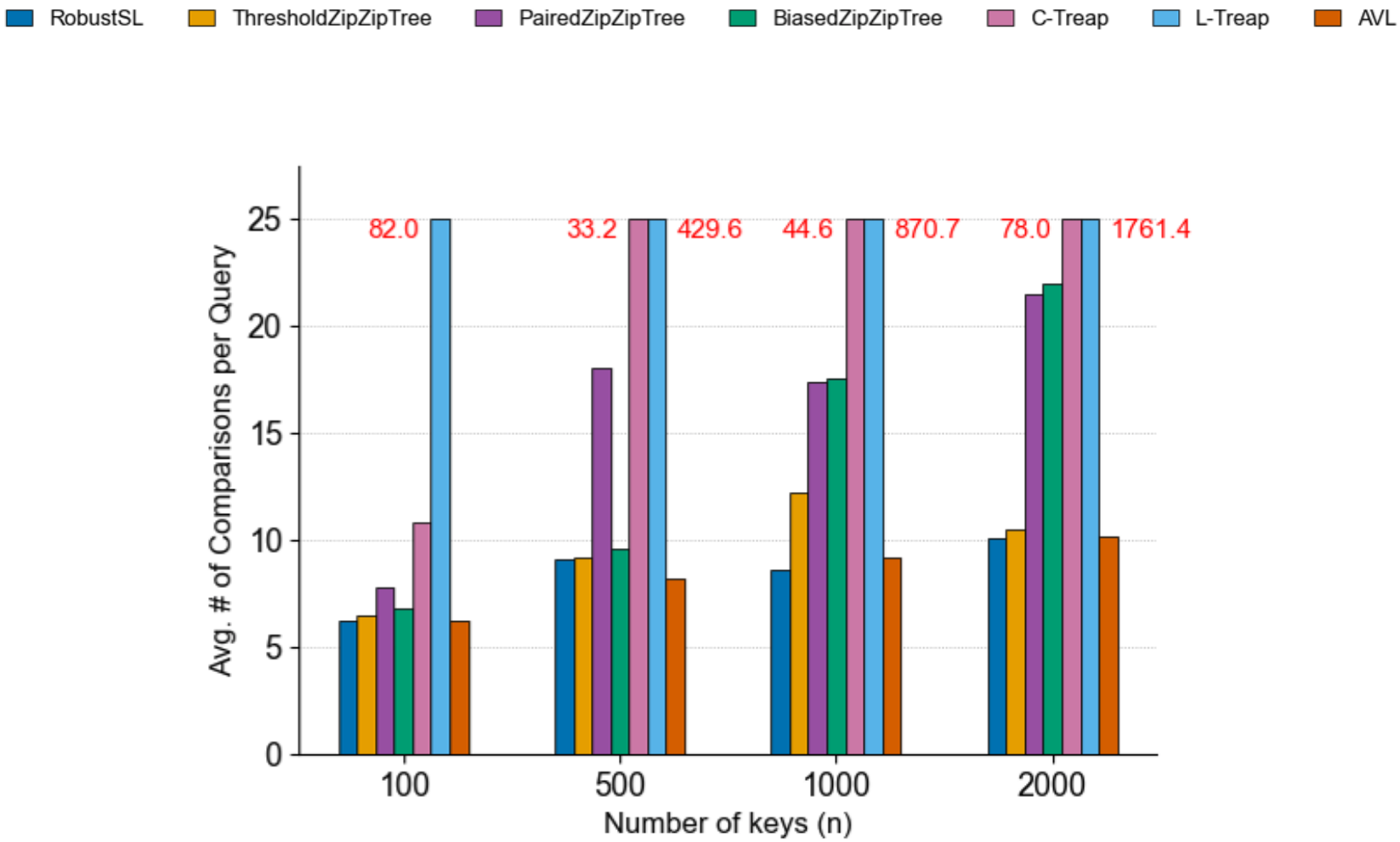
Zipf Parameter Test ($n = 2000,\ \delta = 0$)

Noisy Zipfian Test



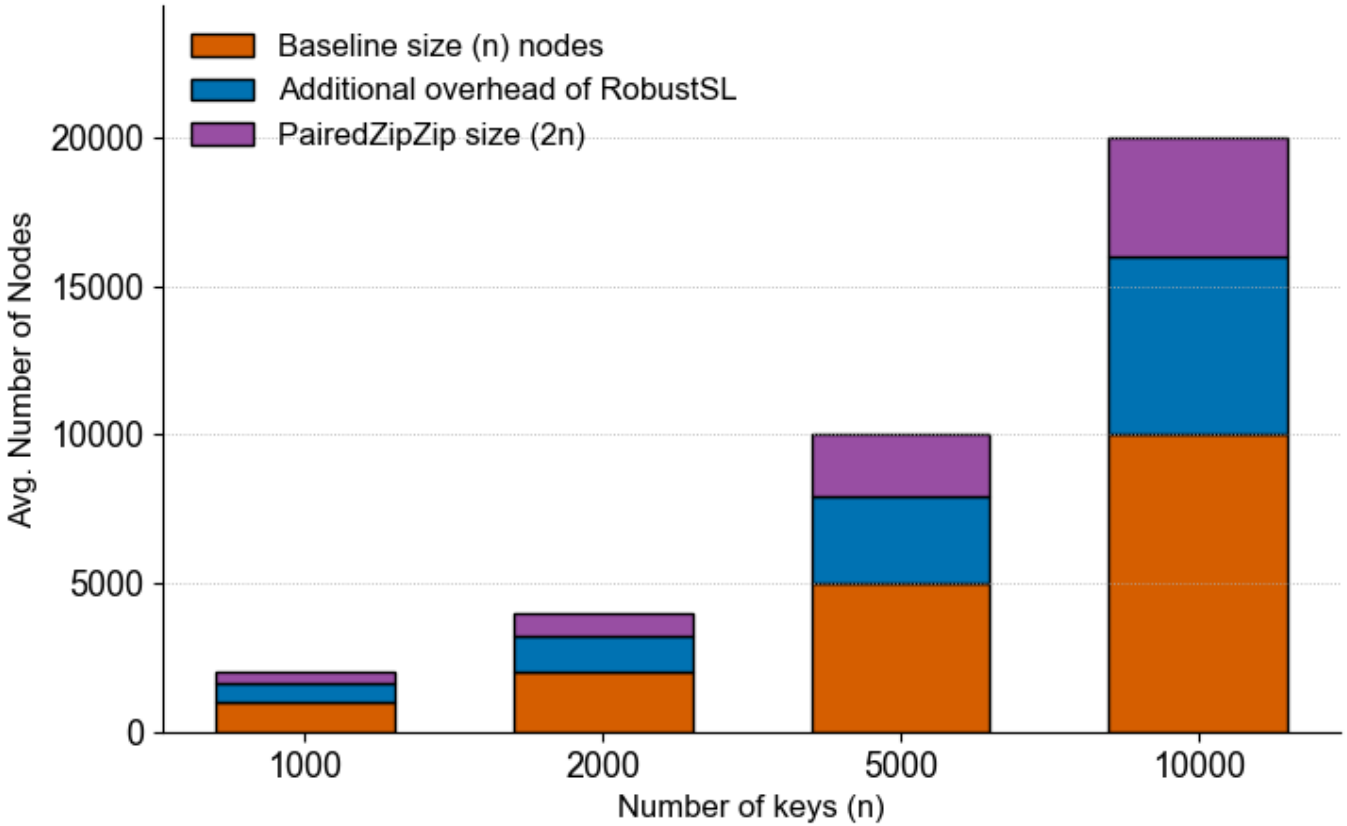
Noisy Zipf ($\alpha = 2,\ \delta = 0.9$)

Inverse Power Test



Inverse Power ($\alpha = 1.01,\ \delta = 0.9$)

Space / Size test



Space usage comparison (Zipfian, $\alpha = 2$). Paired DS uses $\approx 2n$ nodes. RobustSL has skip-list overhead $\approx 1.5\times$.

Experimental Results

- **Thresholded zip-zip**: within 2 comparisons of RobustSL, but uses $\approx 2/3$ the space.
- **Paired zip-zip**: strong H.I., query cost within $\leq 2\times$ of RobustSL for rare keys; uses $\approx 2n$ nodes.
- **Non-robust learned DS** (biased zip-zip, L-Treap, C-Treap): degrade sharply under noisy/inverse-power workloads (can be linear).
- **Bottom line**: Thresholding gives a simple, practical robustness fix; Pairing enforces stronger privacy at a predictable cost.

Space vs Privacy vs Performance

Structure	Consistency	Robustness	H.I.	Space
AVL (classic)	no	yes	none	n
Baseline (learned)	yes	no	none	n
RobustSL	yes	yes	weak	$\approx 1.5n$
Thresholded zip-zip	yes	yes	weak	$\approx n$
Paired zip-zip	yes	yes	**strong**	$\approx 2n$

Conclusion & Next Steps

- Introduced two simple, orthogonal, and practical techniques: **thresholding** and **pairing**.
- **Thresholding**: Makes any consistent LADS robust with very low implementation cost.
- **Pairing**: Strong privacy guarantee at predictable space/time cost.
- **Flaws:** Both our Weak H.I. Update Scheme and Pairing don't give good update times.
- **Future work**: Find a way to improve update times, dynamic experiments, optimize γ, for Paired LADS, Real-World workload Evaluation.

References

[1] H. Lin, T. Luo, and D. Woodruff, "Learning augmented binary search trees," in International Conference on Machine Learning, pp. 13431–13440, PMLR, 2022.

[2] A. Zeynali, S. Kamali, and M. Hajiesmaili, "Robust learning-augmented dictionaries," in International Conference on Machine Learning, pp. 58470–58483, PMLR, 2024.

[3] J. D. Hartline, E. S. Hong, A. E. Mohr, W. R. Pentney, and E. C. Rocke, "Characterizing history independent data structures," Algorithmica, vol. 42, no. 1, pp. 57–74, 2005.

Thank You!

Any Questions?